



## FULLSTACK ENGINEER ASESSMENT TEST

### GENERAL REQUIREMENTS

These are the general requirements that you must follow:

1. You may use any of the following languages: PHP.
2. Your work must be submitted in the form of the source code that is located in a public Git repository.
3. Your code must be clean, concise, properly commented, and easily readable.
4. Bonus points are awarded for:
  - Meaningful commit messages.
  - Publicly accessible API.

### TASK 1: ONLINE STORE

Build an API by following the business requirements below:

1. An **Order** consists of at minimum one **Order Item**.
2. The online store will hold a flash sale, in which customers will try to buy a **Product** at a heavily discounted price.
3. The system must be able to prevent a negative **Inventory** quantity value.

#### Technical requirements:

1. The solution **must** be in the form of an API containing as many endpoints as needed.
  - a. The API **must** use JSON as its message format.
  - b. The API **must** use proper response codes and error messages.
2. The solution **must** demonstrate its ability to handle a race condition during a flash sale, in which a burst of **Orders** will be created and all of them will try to buy a specific **Product**.
3. The solution **must** contain at least one functional test that can be run from the command line, which tests the API's ability to handle a race condition.





### TASK 2: HIDDEN ITEM

Build a simple command-line program for a hidden item game that satisfies the following requirements :

1. Build a simple grid with the following layout:

```
#####  
#.....#  
#...#...#  
#...#...#  
#X#...#  
#####
```

- # represents an obstacle.
  - . represents a clear path.
  - X represents the player's starting position.
2. An item is hidden within one of the clear path points, and the player must find it.
  3. From the starting position, the player must navigate in a specific order:
    - a. Up/North A step(s), then
    - a. Right/East B step(s), then
    - b. Down/South C step(s).
2. The program must output a list of probable coordinate points where the item might be located.
  3. Bonus points : display the grid with the probable item locations marked with a \$ symbol.

